

Cross Validation and Hyperparameters Optimization

Gian Marco Miccio

ReLU — NTNU

07/10/2025

Two things we're going to see now

- Hyperparameters Optimization
- Cross Validation

No heavy Math from now



First, what is a Hyperparameter?

- A **parameter** is a value that the model learns from the training data during the fitting process
- A **hyperparameter** is a value that is set before training and cannot be learned directly from the data

A **parameter** is a value that the model learns from the training data during the fitting process

Examples: In a tree model

- splits for each decision
- values stored in the leaf nodes

In linear regression

- the coefficients β that we look for

A **hyperparameter** is a value that is set before training and cannot be learned directly from the data

Examples:

In a tree model

- Number of trees in the forest
- Maximum depth of each tree
- Minimum samples required to split a node
- Minimum samples per leaf
- Number of features considered at each split

In linear regression

- alpha exponent for Ridge/Lasso



Hyperparameters are important because they produce different models with different performance, and they can be **optimized**

Hyperparameters Optimization: the process of selecting hyperparameters that maximize performance on unseen data

There's different kinds you can do, and the most common are:

- Grid Search
- Random Search
- Bayesian Optimization

Grid Search: select a small number of hyperparameters to try with different values, and then try all combination

Example: For a Random Forest model

- Number of trees: $n_estimators = [100, 200, 300]$
- Maximum depth: $max_depth = [5, 10, 15]$
- Minimum samples per leaf: $min_samples_leaf = [1, 2]$

This produces $3 \times 3 \times 2$ possible combinations!

Important: things can get computationally expensive if you try a lot of values, so stick to a few!



Code for Grid Search

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV

rf = RandomForestClassifier(random_state=42)

param_grid = {
    'n_estimators': [100, 200, 300],
    'max_depth': [5, 10, None],
    'min_samples_leaf': [1, 2, 4]
}

grid_search = GridSearchCV(estimator=rf, param_grid=param_grid, scoring='accuracy')

grid_search.fit(X_train, y_train)
```

Random Search: provide a range for the hyperparameters to pick random combinations from

Example: For a Random Forest model

- Number of trees: $n_estimators \in [100, 500]$ (random integer)
- Maximum depth: $max_depth \in [5, 50]$ (random integer)
- Minimum samples per leaf: $min_samples_leaf \in [1, 5]$ (random integer)
- Maximum features per split: $max_features \in [0.3, 1.0]$ (random float fraction of features)
- Bootstrap: $bootstrap \in \{\text{True}, \text{False}\}$ (random choice)

Decide on how many combinations to try (eg: 20) and then pick the best one



Code for Random Search

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint

param_range = {
    'n_estimators': randint(100, 500),
    'max_depth': randint(5, 50),
    'min_samples_leaf': randint(1, 5),
    'max_features': ['auto', 'sqrt', 'log2'],
    'bootstrap': [True, False]
}

random_search = RandomizedSearchCV(estimator=rf, param_distributions=param_range,
                                   n_iter=10, scoring='accuracy', random_state=42)

random_search.fit(X_train, y_train)
```

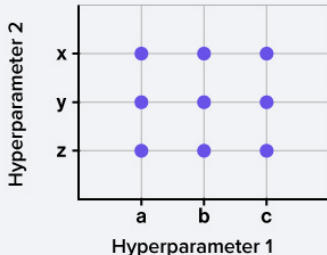


Grid Search

Hyperparameter_One = [a, b, c]

Hyperparameter_Two = [x, y, z]

Hyperparameter_X = [i, j, k]

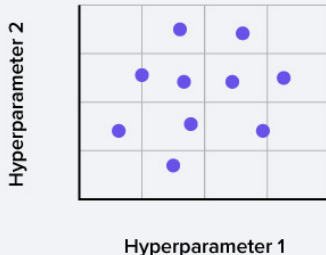


Random Search

Hyperparameter_One = random.num (range)

Hyperparameter_Two = random.num (range)

Hyperparameter_X = random.num (range)



Bayes Optimization: picks combinations by itself like Random Search, but it does by creating a probabilistic model to pick them

Note: it does everything automatically! but it requires a little more set up and different libraries in python (Optuna or Hyperopt)



Let's start now by exploring some questions

- How can you be sure that your model is reliable?

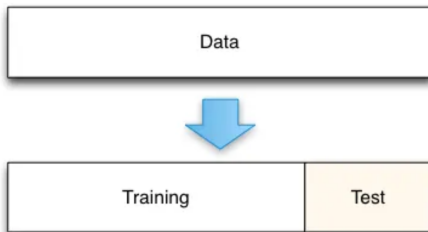
- How can you be sure that your model is reliable?
- and another question:

- How can you be sure that your model is reliable?
- How are you sure that the model you choose is really the best one?

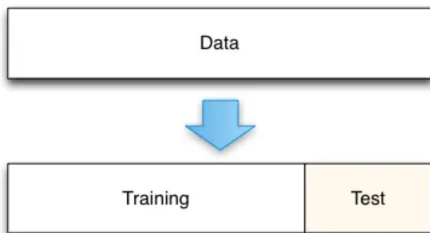
- How can you be sure that your model is reliable?
- How are you sure that the model you choose is really the best one?

These are not questions that have an immediate solution, however we can start with something intuitive that we all are familiar with

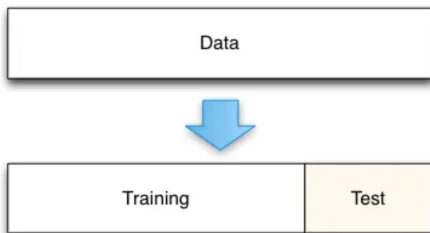
Creating a **test set**



Idea: you pick at random a portion of the dataset (usually around 20%) of the dataset to use as a test, then you calculate evaluation metrics on that



Idea: you pick at random a portion of the dataset (usually around 20%) of the dataset to use as a test, then you calculate evaluation metrics on that



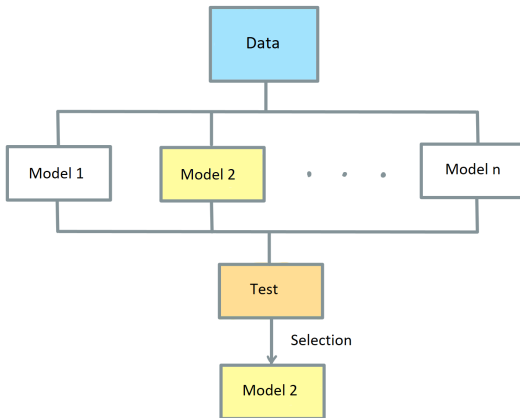
but there's a problem, and it's not an obvious one

Problem: we cannot use the results of the models on the test set to make model selection at the same time

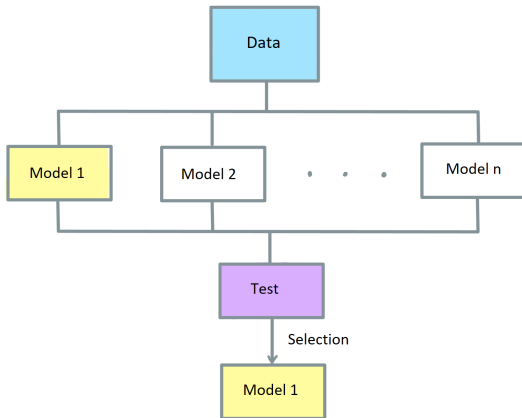
Problem: we cannot use the results of the models on the test set to make model selection at the same time

but why?

Say we have n different models, we train them on the data, then we test them and pick the one with best performance



But if we were to choose a different test set, because we are testing so many models and the differences might not be high, **the result might change**



We call this issue **bias in the test set**

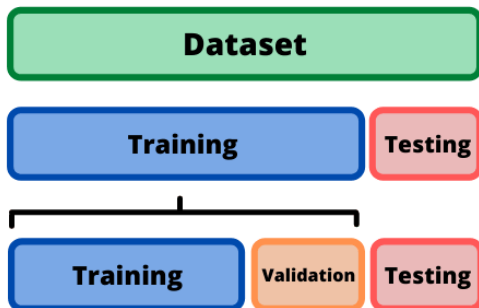
Key takeaway

Model selection and Model test cannot be done on the same dataset, otherwise you'll encounter the problem of bias in the test set

So, what do we do?

We take another portion of the dataset and we use it for model selection, we call this a **Validation set**

Validation set: a subset of the data used to evaluate a model during training and to tune its hyperparameters or make design choices, **without touching the test set**

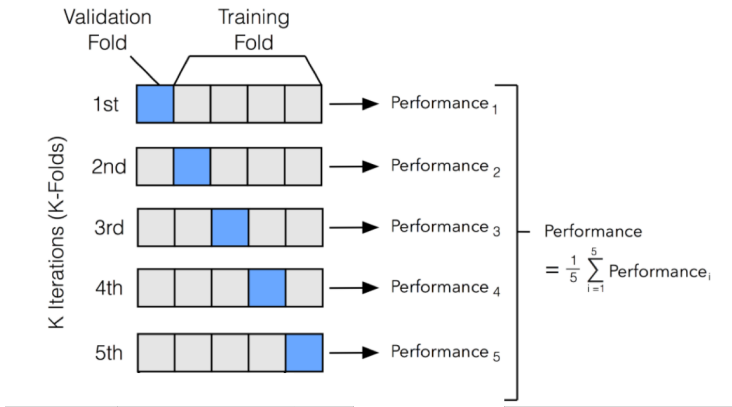


You might think: wouldn't this still create some bias on the validation set like it did before for the test set?

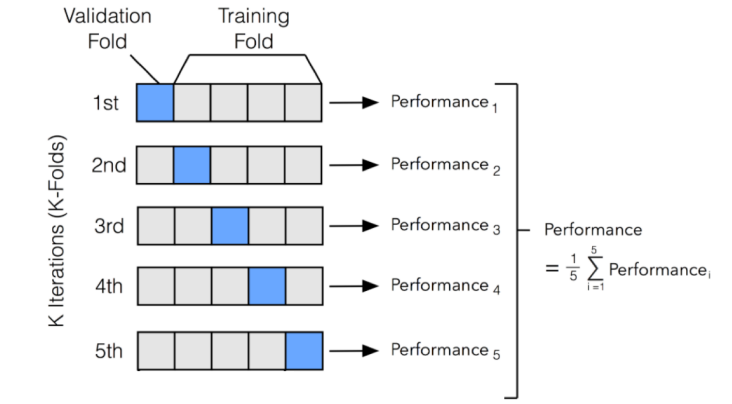
You might think: wouldn't this still create some bias on the validation set like it did before for the test set?

Well yes actually, and that's why we introduce something called **Cross Validation**

Cross Validation: split the dataset into multiple subsets called **folds**, the model is trained and validated multiple times, each time using a different fold as the validation set and the remaining folds as the training set



Cross Validation: The performance scores are then **averaged** to obtain a more reliable estimate of how the model generalizes to unseen data



there's different types of Cross Validation (CV), some more common than others, but they all tackle different cases

- K-fold CV
- Leave One Out (LOOCV)
- Stratified CV
- Time Series CV

K-fold CV: divide the data into k equally sized folds, then use all of them one at a time for validation

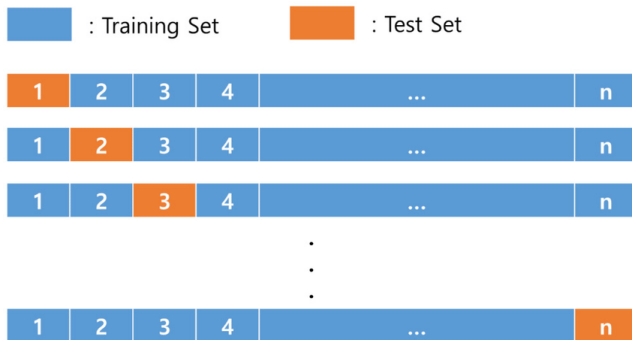
It's the most common!

4-fold validation ($k=4$)



- Pros: very efficient
- Cons: might not be representative of the whole dataset, hence has some variability

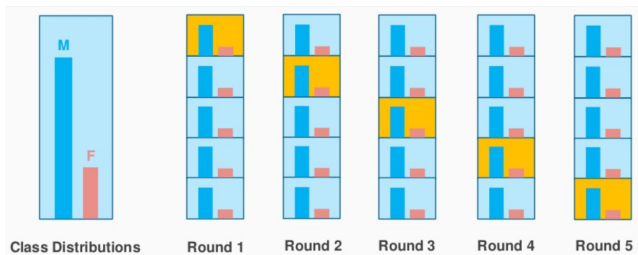
LOOCV: extreme case in which $k=n$, the validation set is only one datum



- Pros: uses all training data
- Cons: very computationally expensive and has high variance

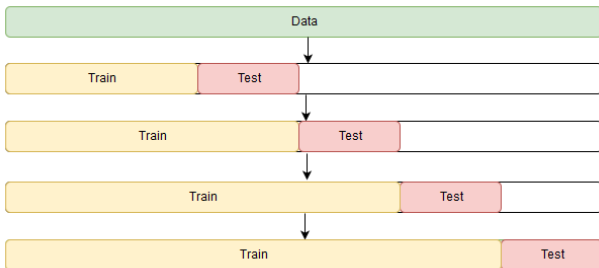


Stratified CV: used when there's high imbalance in classes in the dataset



- Pros: efficient
- Cons: risk that the folds are still not representative

Time series CV: only way to do cross validation on a sequential ordered dataset



- Pros: prevents "future data" from influencing the past
- Cons: has few data to train at the beginning

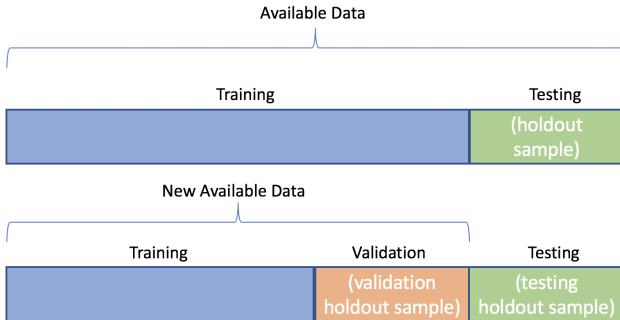
Something very nice is that the functions for hyperparameter optimization contain already an automated cross validation option, and by default it's a good idea to do it

```
grid_search = GridSearchCV(estimator=rf, param_grid=param_grid,  
                             scoring='accuracy', cv=5)
```

one last cautionary tale about **Data Leakage**

Data leakage occurs when information from outside the training set “leaks” into the model during training in a way that shouldn’t be possible

for example, data from the **full dataset** leaking into the **reduced training dataset** after the introduction of a validation set (through cross validation)



we don't want any information from the validation set to influence the training, so for example:

- if we **encode** categorical features, we need to do it inside Cross Validation (if you need the average of the values you don't want to include the validation ones)
- if we **scale** features based on max and min, the validation set might change this range
- if we **select features** you need to do it only based on the training data, not the validation ones as well



For this reason, often times in python you use an object called a **Pipeline**, which groups together every action you want to execute on the training set before training the model **inside** Cross Validation

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from category_encoders import TargetEncoder
from sklearn.model_selection import cross_val_score

pipeline = Pipeline([
    ('encoder', TargetEncoder()),
    ('scaler', StandardScaler()),
    ('rf', RandomForestClassifier())
])

scores = cross_val_score(pipeline, X, y, cv=5, scoring='accuracy')
```

